CSC-223
Assignment 2
Work in a Pair

# Hierarchical Symbol Table

This assignment builds on your previous doubly linked list implementation to create a dictionary data structure we refer to as a symbol table. Your task is to implement a hierarchical symbol table using two parallel doubly-linked lists (one for keys and one for values), with the addition of parent-child relationships to support shadowing and nested scopes in the DEC language.

## What You Need to Do: Continuing From the Last Assignment

You may continue to work with your partner via the same repository or a new repository for this assignment; the same repository is recommended. Please see the "Git and Github Collaboration" section in the first assignment for details.

## What You Need to Do: Adding to the Project Structure

As shown in Figure **1**, place the `SymbolTable.cs` files in the `src/Utilities/Containers` folder as well as corresponding test file in `src/Utilities/Containers.Tests`.

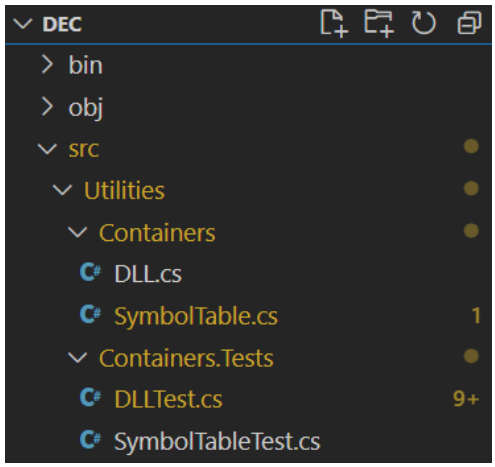


Figure 1: The desired project structure including the `SymbolTable` and its corresponding tests.

## Symbol Tables

A symbol table is a crucial data structure in compiler implementation that maintains a record of all identifiers and their associated information, including variable names, values, data types, and scope details. During name analysis, the symbol table enables efficient identifier management by associating each identifier with its declaration information (such as type, scope, and memory location), allowing the compiler to validate references to previously processed identifiers. Symbol tables manage scope by determining which variables are accessible at different points in the program, preventing naming conflicts and enforcing proper variable visibility rules. (Honestly, that last paragraph is rather daunting since it contains many words and phrases that may not be familiar to you. Come back to this paragraph after having reviewed the examples.)

Our goal is not a full-fledged compiler so our attention will focus mainly on variable scope and some code optimizations.

## A Non-Nested Example: Introduction to *Def-Use*

In this and the forthcoming assignments we will implement support for compilation of a simple, declarative language we will refer to as DEC. A symbol table is a core structure in any compiler for tracking variables and their values during compilation.

In the following DEC code segment, we can observe that each statement follows the DEC language syntax that will be described more thoroughly in a future assignment. In particular, we observe an assignment operator (`:=`), required parentheses around expressions, and operators (exponentiation `**`, addition `+`, multiplication `*`, integer division `//`, and modulus `%`).

```
a := (2 ** 4)
b := (a + 5)
c := ((a * b) // 3)
return (c % 7)
```

The first statement [`a := (2 ** 4)`] assigns the result of 2 raised to the 4th power (16) to variable `a`. The next statement [`b := (a + 5)`] adds `5` to the current value of `a` and assigns the result (21) to variable `b`. The third statement [`c := ((a * b) // 3)`] multiplies `a` by `b` (to get 336), then performs integer division by `3`, resulting in 112 being assigned to `c`. The last statement [`return (c % 7)`] computes `c` modulo `7` returning the value `0` as the result of the program.

The previous description refers to a semantic interpretation and execution of the corresponding code. Execution is not our goal here (nor will it be for the entire course). During compilation, our goal is to populate our symbol table with variable ***definitions*** as they are encountered. When compiling this DEC code, the compiler identifies three variable definitions (`a`, `b`, and `c`) that must be entered into the symbol table. Each time an assignment statement is processed, the compiler adds the variable name on the left side to the symbol table (if it does not already exist). ***Uses*** of variables (those appearing in expressions like the right side of assignments or return statements) would be verified against the symbol table in a later compilation pass.

## A Nested Scope Example

Understanding variable scoping and shadowing is essential for mastering modern programming languages and avoiding subtle bugs in your own code. The following example illustrates how variables with the same name can coexist at different scope levels and how their visibility changes as program execution moves through nested blocks. Consider the following code segment consistent with DEC syntax.

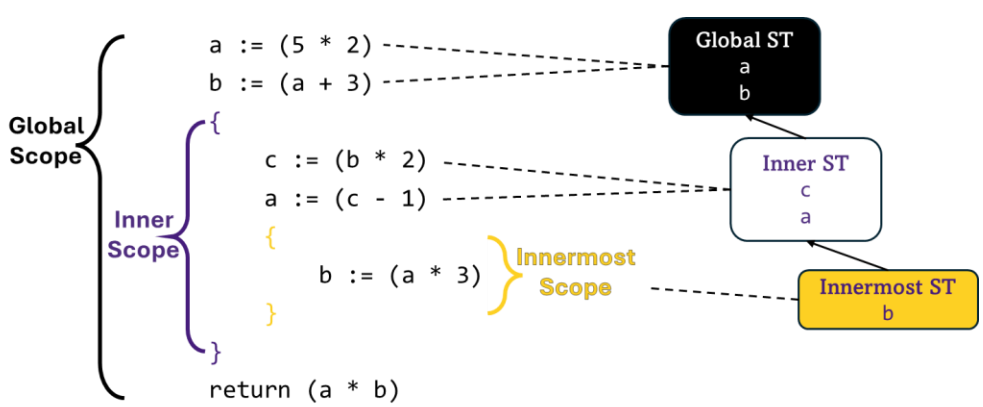


Figure 2: A DEC code segment with nested scoping.

In DEC, as demonstrated in the example in Figure 2, we can create a new variable scope block starting with '`{`' and ending with '`}`'. The code in Figure 2 requires three distinct symbol tables organized in a nested relationship. The global symbol table serves as the outermost scope, containing the initial definitions of `a` and `b`.

After the first two statements are analyzed, the first nested block (labeled as 'inner block') results in a new scope and thus a new symbol table (with pointer to the global symbol table as parent) that will contain its own definition of `a` that temporarily ***shadows*** the global `a`, along with a new variable `c`. Shadowing occurs when a variable declaration in an inner scope creates a new variable with the same name as one in an outer scope, temporarily hiding the outer variable until the inner scope ends. Observe that this inner scope does not contain its own definition of `b`, thus any references to `b` within this inner scope are resolved by looking up the chain to the definition in the global symbol table.

Similarly, the innermost block results in another symbol table with "Inner" as its parent. The innermost table will store a definition for `b` that *shadows* the global `b`. Note that this innermost table can access variables from both the inner scope and the global scope through a chain of lookups: first checking its own table, then the inner table, and finally the global table.

When executed, as a scope terminates, its corresponding symbol table is discarded, restoring visibility to any shadowed variables from outer scopes.

## What You Need to Do: `SymbolTable` Class

For this assignment, you are to implement a symbol table class `SymbolTable<TKey, TValue>` that uses two parallel doubly linked lists (`DLL` objects): one for keys and one for values. Your implementation should provide functionality consistent with the C# dictionary class. The `SymbolTable<TKey, TValue>` class will implement the `IDictionary<TKey, TValue>` interface to ensure compatibility with the C# collections ecosystem. This means that your implementation must include all methods to satisfy the requirements of the `IDictionary` interface.

You will need to implement and test all methods and properties described in

Table 1.

Table 1: Properties and methods for the `SymbolTable` class that fall outside of the `IDictionary` interface.

| Type | | Property | Description |
|---|---|---|---|
| `SymbolTable<TKey, TValue>` | | `Parent` | Read-only property that returns the parent symbol table for accessing higher scopes in the hierarchy. |
| | | | |
| **Return Type** | **Method** | **Description** | **Exceptions Thrown** |
| `None` | `SymbolTable(SymbolTable<TKey, TValue> parent)` | Initializes a new symbol table with a parent reference, creating a scope hierarchy relationship. | |

| | | | |
|---|---|---|---|
| bool | ContainsKeyLocal(TKey key) | Checks for key existence only in the current scope, not traversing up to parent scopes. | Throws an ArgumentNullException when the key is null. |
| bool | TryGetValueLocal(TKey key, out TValue value) | Attempts to retrieve a value only from the current scope without searching parent scopes. | Throws an ArgumentNullException when the key is null. |

***Facilitating shadowing.*** Consider another example DEC program using a combination of shadowing and reference to an outer scope that is more likely to occur in practice.

```
{
    x := (10)
    {
        y := (20)
        x := (x + y)
    }
    return (x)
}
```

In this example, the inner scope defines `x` that shadows the `x` in the outer scope. However, in the assignment statement defining `x`, the `x` in the outer scope is being referenced. To facilitate (1) using a variable in an inner scope and (2) shadowing that variable after a use, the `TryGetValue` implementation for `SymbolTable` must account for this situation. If a variable is registered in a `SymbolTable` (meaning its key exists) but has not been assigned a value yet (its value is `null`), the system will check its parent symbol table to find an existing meaningful value for that variable.

## What You Need to Do: Testing

All tests should be xUnit tests and adhere to the norms of `xUnit` including using `[Fact]` and `[Theory]`.

***New to CSC-223!*** LLMs can be helpful tools for generating testing code for your assignments. When using an LLM to create tests, include prompts like "write thorough tests for all methods", "make sure to test with duplicate data as well" to ensure comprehensive coverage, and "use `[Theory]` and `InlineData` to avoid redundancy in testing code". ***However, be warned that generated code isn't always perfect—your professor has encountered several cases where tests were written incorrectly for the corresponding code.*** Always review each test carefully to verify it is testing what you intend and functioning properly with your implementation. It is also fair to ask the LLM to see if any cases have been missed in the tests that it generates. In fact, it is advised that you ask an LLM to 'check its work'; do not accept a first answer.

Even though you may use an LLM for testing, you will explicitly be asked about testing shadow-based functionality.

## What You Need to Do: Commenting

Comment code consistent with the standards stated in the first assignment: when needed, use an LLM to generate comments then refine as needed.

## Submitting: Source Code

You will demonstrate your source code to the instructor, in person. Be ready to demonstrate successful execution of all unit tests and describe all tests and code you were required to implement.